

Attention Is All You Need

20220798 정석환

목 차

01

연구 배경

02

모델 개요

03

Self-Attention

04

Multi-Head Attention

05

Position-wise Feed
Forward Network

06

Positional Encoding

07

Residual Connection +
Layer Normalization

08

실험 결과

09

결론 및 영향

연구 배경

기존 문제점

- RNN : 순차적 처리 → 병렬화 제약
- 긴 문장 → 장기 의존성 학습 어려움

해결 아이디어

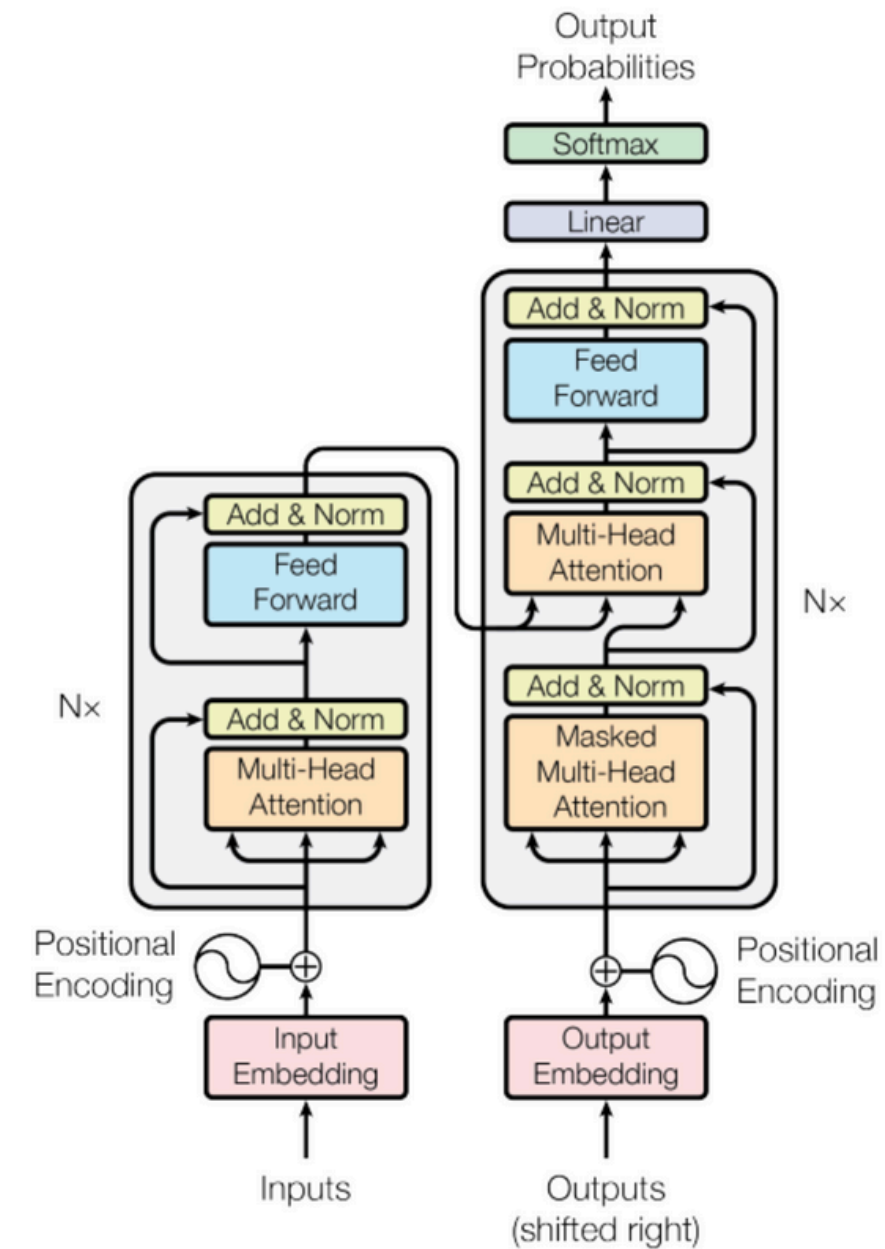
모든 입력 토큰 간 관계를 Recurrence나 Convolution 없이 Attention만으로 계산

Transformer 모델 개요

구조

Encoder-Decoder 구조 유지

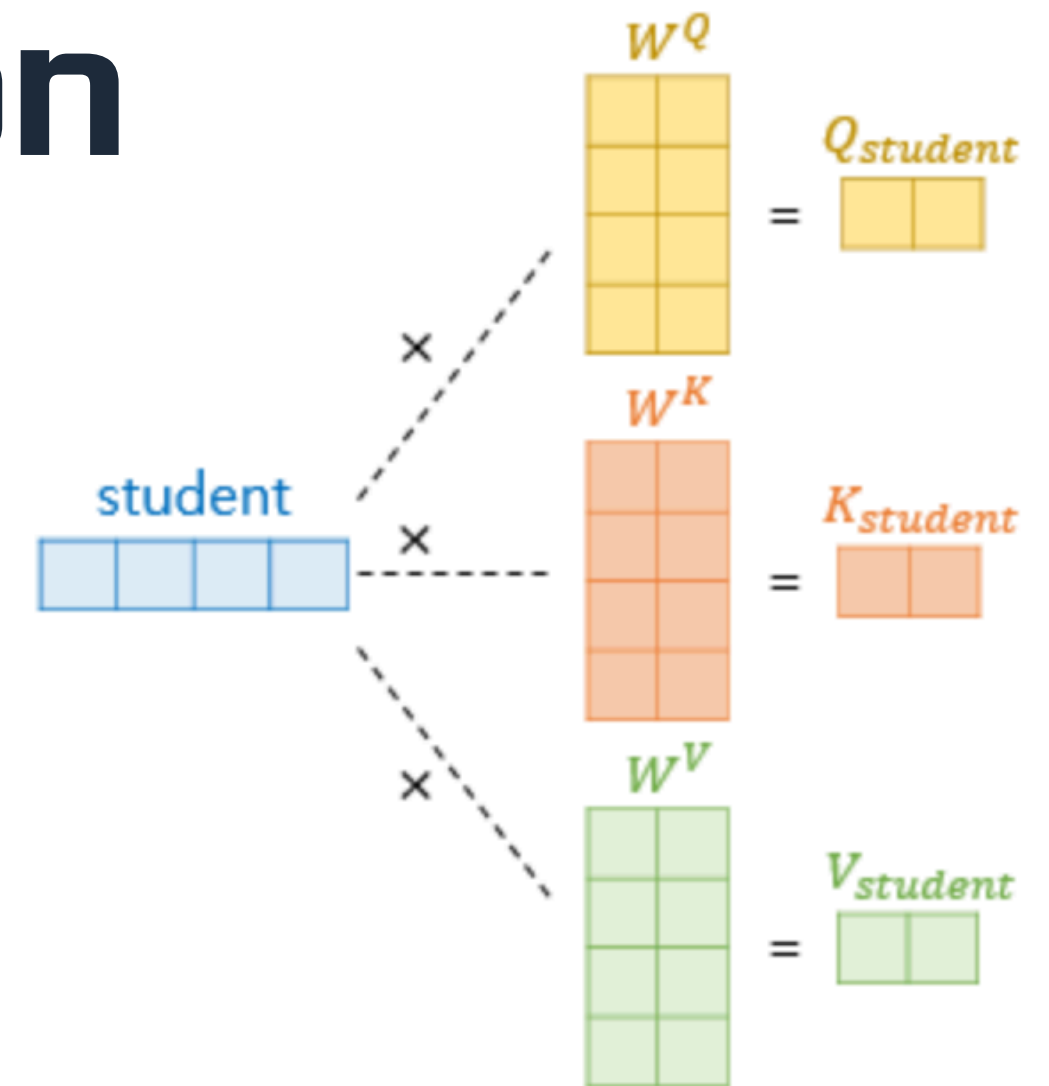
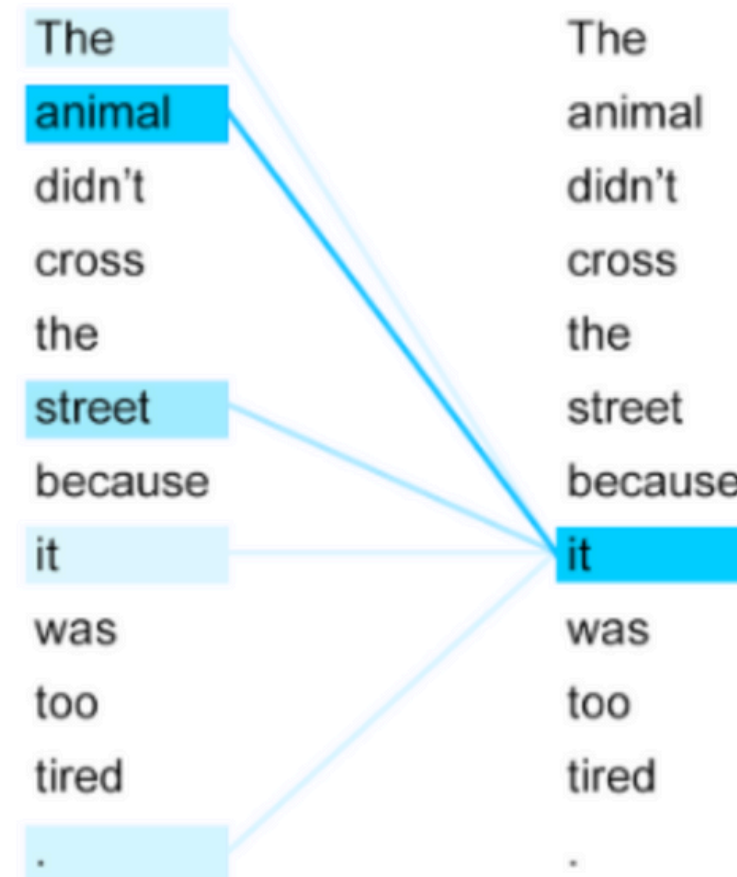
모든 연산을 Self-Attention 기반으로 대체



Self-Attention

Self-Attention

하나의 시퀀스 안에서
각 단어가 서로를 참고하며
관계를 학습



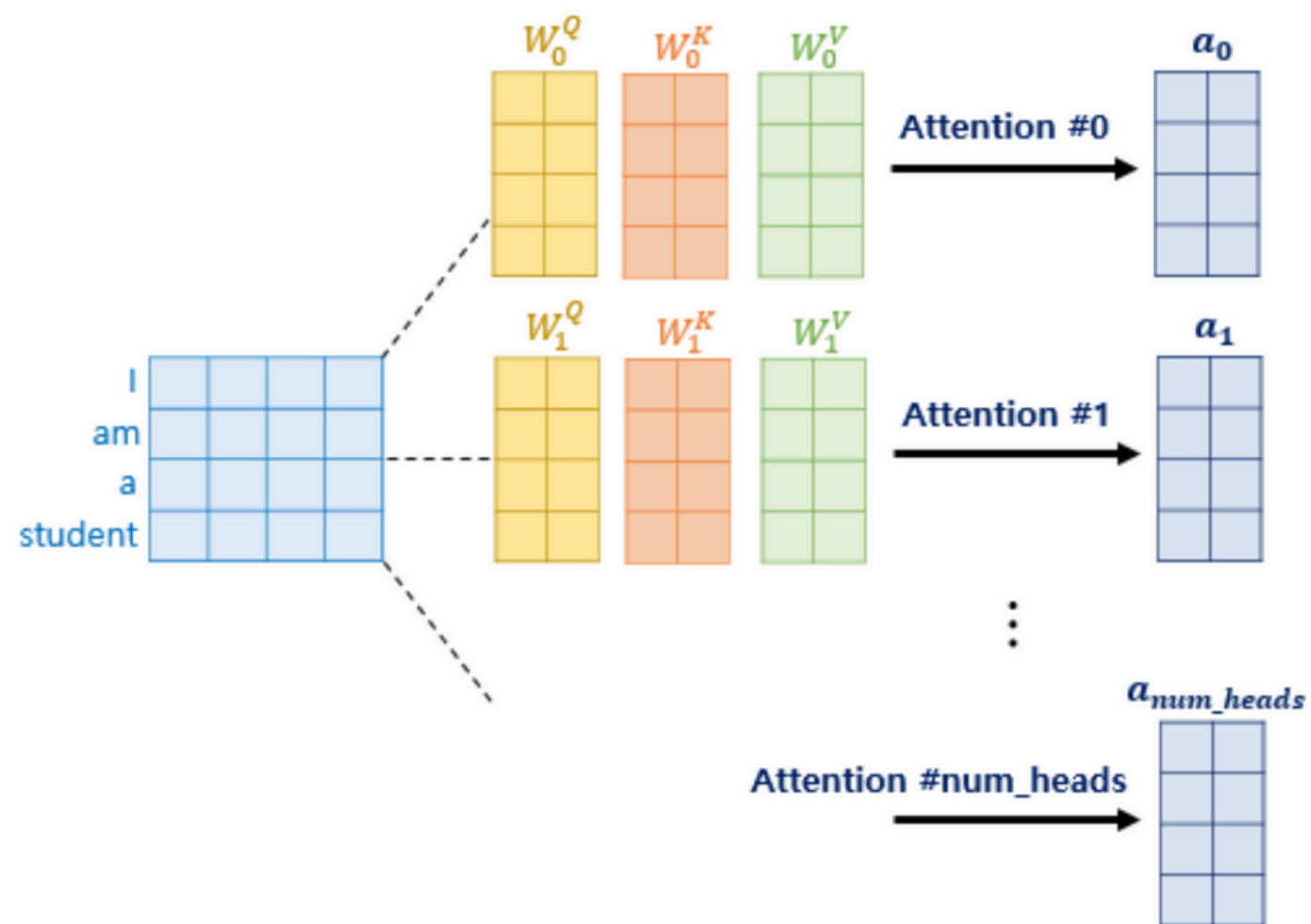
Query, Key and Value

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention

Multi-Head Attention

Multi-Head Attention은
여러 개의 Self-Attention을 병렬로 수행
해서
서로 다른 관점(문법, 의미, 위치 등)
에서 정보를 학습하는 구조



Multi-Head Attention

Which **do** you like better, coffee or tea?

- 문장 타입에 집중하는 어텐션

Which do **you** like better, **coffee** or **tea**?

- 명사에 집중하는 어텐션

Which do you **like** better, **coffee** or tea?

- 관계에 집중하는 어텐션

Which do you like **better**, coffee or tea?

- 강조에 집중하는 어텐션

하나의 Attention Head는 한 가지 관계만 본다

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

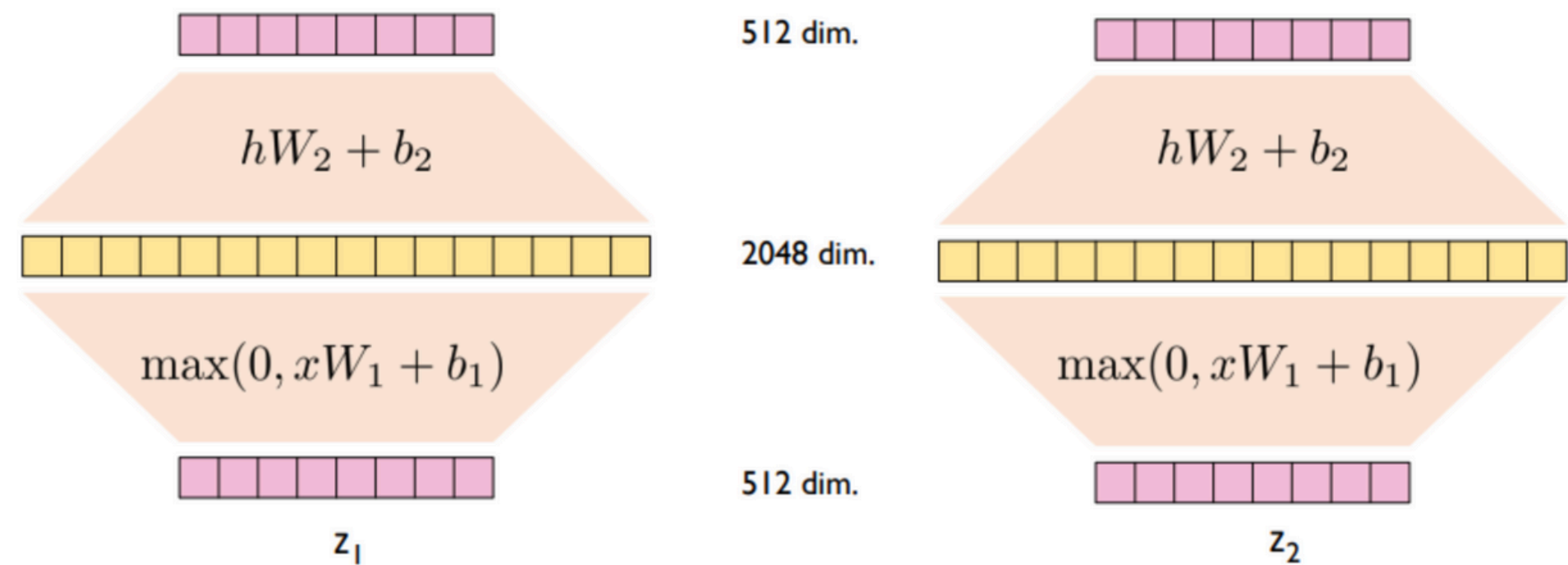
1. 원래 Query, Key, Value 행렬 값을 head 수만큼 분할
2. 분할된 행렬 값을 통해, 각 Attention value값들을 도출
3. 도출된 Attention value값들을 concatenate(쌓아 합치기)하여 최종 Attention value도출

Position-wise Feed Forward Network

Position-wise Feed Forward Network

FFN은 각 단어 벡터를 독립적으로
512→2048→512
변환시키는 작은 신경망

Self-Attention 이후 각 단어의 의미 벡터를 강화하는 역할



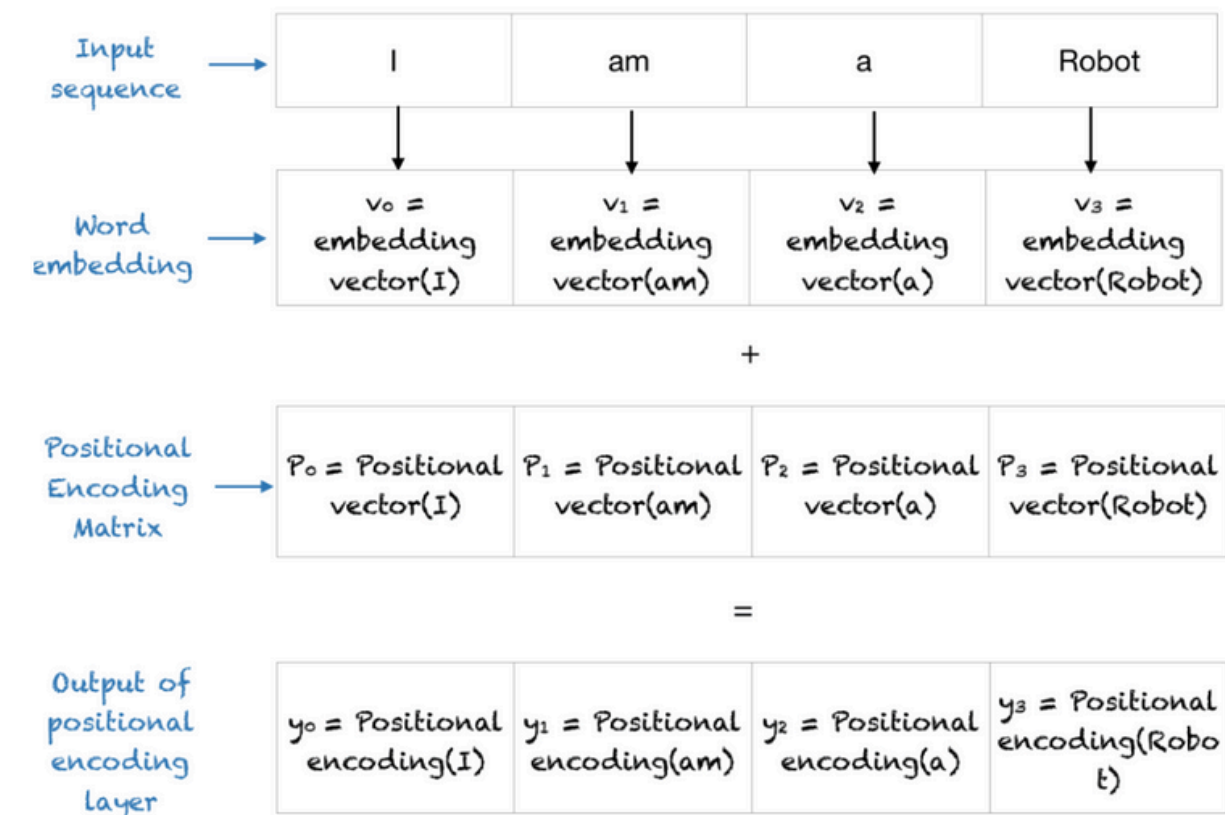
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Positional Encoding

Positional Encoding

transformer는 전체 정보를 통으로 입력 받기 때문에 위치 정보가 손실 된다

이를 해결하기 위해 positional encoding을 통해 위치 정보를 어느 정도 보존한다.



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

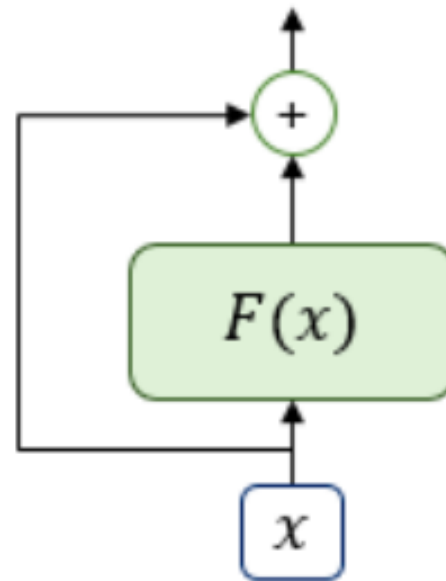
Residual Connection & Layer Normalization

Residual Connection & Layer Normalization

Residual Connection
: 이전 입력을 그대로 다음 층에 더해줘
정보가 사라지지 않게 하는 방식

Layer Normalization
: 각 층의 출력을 정규화해서 학습을 안정
화시키는 과정

$$H(x) = x + F(x)$$



Residual Connection

$$LN = LayerNorm(x + Sublayer(x))$$

Layer Normalization

실험 결과

실험 결과

Transformer가 기존의
RNN이나 CNN 기반 번역 모델보다
훨씬 높은 번역 품질을 내면서,
연산량은 훨씬 적게 든다는 걸 보여줍니
다.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

결론 및 영향

Transformer는 RNN이나 CNN 없이
오직 Attention 메커니즘만으로 작동하는 최초의 모델

Self-Attention 구조를 통해 병렬 학습이 가능해졌고,
장기 의존성(Long-range dependency) 문제를 해결

딥러닝 아키텍처의 새로운 표준을 제시

이후 대부분의 대규모 AI 모델의 기반 구조가 Transformer로 전환

참고문헌

https://ratsgo.github.io/nlpbook/docs/language_model/tr_self_attention/

<https://lcyking.tistory.com/entry/%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-Attention-is-All-you-need%EC%9D%98-%EC%9D%B4%ED%95%B4>

<https://jaylala.tistory.com/entry/%EA%B0%9C%EB%85%90%EC%A0%95%EB%A6%AC-%EB%A9%80%ED%8B%B0%ED%97%A4%EB%93%9C-%EC%85%80%ED%94%84-%EC%96%B4%ED%85%90%EC%85%98Multi-Head-Self-Attention>

<https://sonstory.tistory.com/89>

<https://wikidocs.net/22893>

Attention Is All You Need

감사합니다